Testing a Conversational Agent

> Part of the Conversational Agent curriculum. This document covers an injectable LLM seam for hermetic tests, stub-based graph-routing tests, testing reducers against a real graph invocation, real-subprocess integration testing of the tool connection, exactly one gated live-LLM smoke test, table-driven tests for the deterministic parts, and answer linting plus a manually-run golden-prompt evaluation.

1. The core problem: LLM calls are slow, costly, and non-deterministic

A normal test suite can't call a real model on every run — it's slow, it costs money per call, and a genuinely non-deterministic response would make otherwise-correct tests flaky for reasons that have nothing to do with a real bug. The fix isn't a testing trick bolted on afterward — it has to be a design decision made up front: **the model itself must be swappable.**

2. An injectable LLM seam

Every LLM-calling node should take the model client as an explicit argument (or obtain it from a small factory function called at graph-build time), rather than importing and constructing a client at module import time:

```
def build_graph(llm):
    def agent_node(state):
        response = llm.invoke(state.messages)
        ...
    graph = StateGraph(AgentState)
    graph.add_node("agent", agent_node)
    ...
    return graph.compile()
```

This is what lets a test hand in a stub or fake model instead of a real one. Set the bar high: the **entire test suite should pass with no API key configured at all.** A suite that silently skips itself whenever a key happens to be absent — or, worse, quietly makes real network calls whenever one happens to be present in the environment — isn't actually protecting anything in CI; it's protecting whatever developer happens to be running it locally with a key set.

3. Stub LLMs for graph-routing tests

For a routing/classification node, inject a canned response object (a fixed classification, or a fixed "call this tool with these arguments" request) and assert on which node the graph transitions to next:

```
class StubLLM:
    def __init__(self, response):
        self._response = response
    def invoke(self, messages):
        return self._response

def test_off_topic_routes_to_off_topic_node():
    app = build_graph(llm=StubLLM(response=FixedClassification("off_topic")))
    result = app.invoke({"current_query": "what's the weather tomorrow"})
    assert result["classification"] == "off_topic"
```

This tests the **graph's own logic** — its edges and conditions — completely independently of whether the real model's judgment would actually classify that query correctly. That's a separate, much narrower question, deliberately isolated to a single gated test (§6).

4. Testing state and reducers with a real graph invocation

A reducer's merge behavior (Core Concepts, §4) is a property of actually running the compiled graph — constructing two state objects by hand and comparing them never exercises the reducer at all, since the reducer only runs as part of a real state update inside the graph's own execution.

```
def test_add_messages_reducer_appends_across_nodes():
    app = build_graph(llm=StubLLM(response=...))
    app.invoke({"current_query": "first question"}, config=THREAD_CONFIG)
    state_after_second = app.invoke({"current_query": "second question"}, config=THREAD_CONFIG)
    assert len(state_after_second["messages"]) > 2   # accumulated, not overwritten
```

Write at least one test like this for any custom or non-obvious reducer — it's cheap to write and it's the only way to actually prove the merge behavior you're relying on elsewhere in the design is real, not assumed.

5. Integration-testing the tool-calling connection against a real server, not a mock

Mocking the MCP client entirely risks missing a genuine wire-level shape mismatch — exactly how a real tool result's content is structured (see MCP Server, Core Concepts, §7's content-block shape) is a property of the real protocol implementation, not something worth guessing at when hand-building a fake. Keep at least one integration test that launches a real instance of the MCP server as a subprocess and exercises the actual client connection against it — slower than a pure mock, but specifically able to catch this class of adapter-shape bug that a hand-built fake, by construction, cannot.

6. Exactly one gated, real-LLM smoke test

Everything else in the suite runs on stubs. Keep exactly one test that makes a genuine call to the real model, gated to skip itself automatically when no API key is configured:

```
import os
import pytest

@pytest.mark.skipif(not os.environ.get("ANTHROPIC_API_KEY"), reason="requires a real API key")
def test_agent_produces_sane_response_for_real_query():
    app = build_graph(llm=get_llm())   # the real client, not a stub
    result = app.invoke({"current_query": "What are the historical trends for a major emitter?"})
    assert result["response_text"]
```

This is the one place that actually answers "does this prompt genuinely produce a sane response from the real model" — a question no stub can answer on its own — deliberately kept to a single test so it doesn't slow down or destabilize the rest of the suite, and deliberately gated so CI environments without a configured key still pass cleanly rather than erroring.

7. Testing guardrail and limit behavior directly, not just end-to-end

For a hard boundary condition — a tool-call cap (Tool Calling and Guardrails, §3), or an input-validation rule on a public-facing parameter like a session/thread identifier — write a **unit test directly against the function enforcing it**, not only an end-to-end test that exercises the whole graph or the whole HTTP endpoint.

An end-to-end test can pass "by accident": the overall observable behavior looks correct while a real gap sits inside the guard function itself (for instance, a freshly generated identifier that skips a registration step the guard was supposed to enforce — invisible from outside, because the identifier still happens to work on its very first use). Test the guard function directly, with inputs chosen specifically to exercise its edge cases, not just its typical path.

8. Testing the deterministic parts: tables, caps, and builders

Tool Calling and Guardrails, §7–§10 moved a lot of behavior out of the model and into plain functions: the required-notes table, the model-view cap, the summary/KPI/source-line builders, the follow-up lookups. That is a testing windfall — all of it is ordinary deterministic code and needs no LLM:

- **Table-driven tests for every lookup.** One parametrized test per table (tool → required notes, tool → links, tool → chips) that asserts the *exact* output for each tool, a failed call contributing nothing, and de-duplication across calls.
- **The cap never mutates its input.** Assert the stored full result is untouched after `cap_for_model`, that first/last points survive, and that summary and envelope blocks pass through byte-for-byte.
- **Units get their own test.** Feed a raw result in a small unit and assert the summary's reporting-unit block — the specific regression "model wrote a number 1000× too big" is prevented by a test on the *builder*, not by a prompt.

- **Pin the cross-repo contracts by reading the other side's source.** A link test that scrapes the frontend's route table, anchor ids, and the URL-parameter names its pages read will fail the day either side renames something. This is cheaper and more reliable than a browser test for the same guarantee.
- **Typography and wording as tests.** If user-facing strings must follow a house style (proper unit subscripts, en dashes instead of `--`), a test that scans titles and notes keeps it from regressing silently; text written for the model rather than the user is exempt.

9. Linting answers, and the manual golden-prompt evaluation

Some failures can only be seen in model *output*: a relationship called by a name reserved for a different, better-validated one; a global outcome attributed to one actor; correlation presented as proof; a quoted figure with no accompanying disclosure; an illustrative number presented as measured. Two tools cover this, neither of which is a normal unit test:

1. **An answer linter** — deterministic checks over a response's text (regexes and small rules), run in unit tests against canned good and bad answers. It must understand negation: "this is *never* called X" is compliant, "this is X" is not. Treat it as a coarse net, not proof: it catches the known failure shapes and says nothing about new ones.
2. **A golden-prompt evaluation script** — a fixed set of prompts (the starter prompts plus deliberate traps: leading questions, requests to attribute, requests for an unsupported option) run against the *real* stack and the *real* model, with every answer passed through the linter and the output reviewed by a person. Exit non-zero if any case is flagged.

Keep the second one **a script you run before deploying a prompt or guardrail change, not a pytest file**. The suite's rule is exactly one real-LLM test; an evaluation that calls a live model ten times is slow, costs money, and is non-deterministic by nature, so it belongs on the release checklist, not in CI. Run it on the model you ship (and say in the docs which models it has and has not been validated on — a smaller local model that passes the tool-calling battery has not thereby passed the guardrail one).

10. A change in the tool server is a change in the agent's test surface

The agent's suite launches a real tool-server subprocess (§5), which means it asserts things like "the server exposes exactly these tools." When the tool server gains or renames tools, the *server's* own suite stays green while the *agent's* quietly goes red — and nobody notices because nobody ran it. Make "run the consuming agent's suite" part of the checklist for any tool-server change, and write that assertion so it compares against the live tool list (a count that is hard-coded in the test's name goes stale the first time it is right).

11. Testing approach by layer

Layer	Test with	Why
Routing/classification nodes	A stub LLM returning a canned response	Tests the graph's own edges, independent of real model judgment

Layer	Test with	Why
Reducers	A real compiled-graph invocation across sequential updates	Merge behavior only exists at graph-execution time
The MCP tool connection	A real MCP server subprocess	Catches wire-level shape bugs a hand-built fake can't
Real model behavior	Exactly one gated, key-conditional test	The one question no stub can answer; kept narrow deliberately
Hard guards/limits (caps, id validation)	A direct unit test on the guard function	End-to-end tests can pass while the guard itself has a real gap
Lookups, caps, summary/KPI builders	Table-driven unit tests, no LLM	Deterministic code; exact outputs can be asserted
Cross-repo contracts (routes, anchors, URL params)	A test that reads the other side's source	Fails the day either side renames something
Answer wording and disclosure rules	An answer linter + a manually-run golden-prompt script	Only visible in real model output; too costly and non-deterministic for CI

See also

- Conversational Agent curriculum index
- Core Concepts and Tool Calling and Guardrails for the mechanisms under test in this document
- MCP Server Testing for the matching testing discipline on the tool-source side of this connection
- Deployment — the boundaries this document's §7 guard tests are meant to protect once the agent is actually public